Medical Robotics — 2026 Final Projects

Detailed Project Descriptions and Guidelines

Note on references. Canonical references are listed at the end. For Projects 13–16 the instructors will additionally provide the specific bibliography, dataset and/or baseline code; the references here are the foundational ones students should read first.

1. Project 4 — Multi-RCM for the ROSA robot

Goal: Implement a double RCM through kinematic control for neurosurgical procedures supported by ROSA.

Description: In the ROSA system for neursurgery the RCM is located at the skull entry point; in some phases of the procedure it would be useful to have the RCM located at the target point and allow small conical motion at the entry point.

Your task:

- implement a 3D and kinematic model of ROSA in Unity;
- implement the double RCM through kinematic control.

Best suited for: students who already have, or wish to develop, a solid background in robot kinematic control.

Learning objectives: building a 3D model of a robotic stìystem; task control with kinemtic constraints.

Suggested approach: search first for an already available 3D model of ROSA or a manipulator with the same kinematics.

References: [12].

2. Project 6 — Deformable blood-vessels model

Goal: A deformable model of blood vessels that runs in real time inside the dVRK (da Vinci Research Kit) surgical simulator.

Description: Realistic finite-element vessel deformation is too slow for an interactive simulator. The project therefore builds a *surrogate* (learned, fast-to-evaluate) model that reproduces the deformation of vessels of standard geometry under tool interaction, and embeds it in the Unity-based dVRK scene.

Your task:

- Model the vessels with standard primitives (e.g. cylinders / generalised cylinders along a centerline).
- Generate ground-truth deformations of these primitives under contact/pushing loads (mass–spring or FEM reference).
- Train a 3D surrogate that maps tool contact (position, force) to vessel deformation, fast enough for real-time inference.
- Embed the trained surrogate in the dVRK Unity simulator and validate visual and force plausibility.

What we provide: dVRK in Unity.

Best suited for: students who already have, or wish to develop, a solid background in robot kinematics and machine learning.

Learning objectives: reduced-order / surrogate modelling of deformable bodies; real-time inference constraints; integration of a learned model into a robot simulator; validation against an FEM reference.

Suggested approach: start from a low-dimensional parametrisation of the centreline and cross-section; learn a deformation operator (see the operator-learning references [6,7] used by Projects 13–15); benchmark inference latency explicitly.

References: [9] (dVRK), [6,7] (operator learning), [10] (MuJoCo for reference dynamics if used).

3. Projects 7, 8, 9, 10 — Dataset generation with FEBio

Goal: Generate interaction datasets for the organs of a virtual abdominal phantom using **FEBio** (Finite Elements for Biomechanics) for neural-network training.

Description: Each project takes one organ of the abdominal phantom and builds a physically realistic finite-element model in FEBio, then runs a campaign of tool–tissue interaction simulations to produce a labelled dataset (applied load / contact → resulting deformation and reaction force). These datasets feed the learning-based soft-body projects (13–15) and the surrogate model (6).

Project split (one organ each):

- **Project 7 — Liver**
- **Project 8 — Kidneys**
- **Project 9 — Skin**
- **Project 10 — Blood vessels**

Your task:

- Build the FEM mesh of the assigned organ and assign realistic constitutive properties (typically nearly-incompressible hyperelastic, e.g. Ogden / Mooney–Rivlin, with literature parameters for that tissue).
- Define realistic boundary conditions and tool–tissue contact.
- Run a parametrised batch of interaction simulations (varying probe pose, contact location, load).
- Export a clean, documented dataset (inputs, deformation fields, reaction forces) suitable for NN training.

What we provide: FEBio workflow guidance; the abdominal phantom geometry.

Best suited for: students in Biomedical Engineering.

Learning objectives: soft-tissue constitutive modelling; finite-element setup and contact; design of a machine-learning dataset (coverage, sampling, train/test split, units and reproducibility).

Suggested approach: fix a common data schema across the four organs so the datasets are interoperable; document the constitutive parameters and their literature source; include a held-out test distribution.

References: [8] (FEBio); tissue-parameter sources to be cited per organ in the report.

4. Project 11 — Simulation model of the brain

Goal: A simulation-ready model of the brain integrated in Unity or Unreal Engine.

Description: Brain tissue is very soft and nearly incompressible and is commonly represented with a hyperelastic (and optionally viscoelastic) constitutive law. The project produces a 3D brain model with physical properties suitable for interactive simulation.

Your task:

- Obtain a 3D brain model (build one, or source an existing anatomically correct mesh) and prepare it (clean, simplify, scale).
- Import it into Unity or Unreal Engine.

- Embed realistic physical properties for soft-body simulation and verify stable, plausible interactive behaviour.

What we provide: preliminary model provided by neurosurgeon (to be confirmed); project supervision; choice of engine left to the student.

Best suited for: all students.

Learning objectives: 3D asset preparation pipeline; mapping tissue mechanics onto a game-engine soft-body / deformation system; the trade-off between physical fidelity and interactive performance.

Suggested approach: document the chosen constitutive assumption (e.g. nearly-incompressible hyperelastic) and its parameters; validate with a simple indentation test before integration.

5. Project 12 — Motion retargeting of hand motion

Goal: Map the motion of the human hand onto the dVRK simulator tools.

Description: Teleoperating the simulated da Vinci instruments from free-hand motion requires a retargeting function from the human hand configuration to the robot tool pose/joint space. The project compares retargeting strategies and adds new instruments to the simulator.

Your task:

- Survey and compare retargeting methods (e.g. direct task-space mapping of the fingertip/grip frame with motion scaling and clutching; joint-to-joint mapping; inverse-kinematics-based task-space retargeting).
- Determine the most effective method for the dVRK tools.
- Add new tools to the simulator and implement the retargeting in Unity.

What we provide: the dVRK Unity simulator.

Best suited for: all students.

Learning objectives: robot kinematics of the dVRK instrument; master–slave motion scaling and clutching; quantitative comparison of retargeting strategies (accuracy, smoothness, user effort).

Suggested approach: define an evaluation protocol (tracking error, jerk/smoothness, task completion) before comparing methods, so the "most effective" claim is measured rather than asserted.

References: the dVRK kinematics in the course slides and the relative scientific paper.

6. Projects 13, 14, 15 — Learning-based soft-body simulation

Goal: Explore different learning-based methods for soft-body simulation and port the resulting model into Unity.

Project split (one method each; methods can be updated at the project starting time, the following are propositions):

- **Project 13 — Causal PINNs.** Physics-informed neural networks with the temporal-causality training reformulation [5], applied to the soft-tissue deformation PDE.
- **Project 14 — Physics-based DeepONet + FEM data.** A (physics-informed) DeepONet [6,7] operator learnt with FEM simulation data (e.g. the FEBio datasets from Projects 7–10).
- **Project 15 — Deformation–force correlation learning.** Learn the deformation–force relation in MuJoCo [10], then port the trained model into Unity.

Your task: implement the assigned method, train it on soft-tissue interaction data, evaluate accuracy and inference speed against an FEM reference, and demonstrate the MuJoCo-to-Unity transfer.

What we provide: coordination with the dataset-generation projects (7–10).

Best suited for: students who already have, or wish to develop, a solid background in machine learning.

Learning objectives: physics-informed learning and operator learning; the accuracy/speed trade-off versus classical FEM; sim-to-engine transfer.

Suggested approach: share a common evaluation harness across 13–15 (same test cases, same error and latency metrics) so the three methods can be compared head-to-head; foundational PINN/DeepONet background in [3,4,6].

References: [3] (PINNs), [4] (DeepONet), [6] (physics-informed DeepONet), [5] (causal PINNs), [10] (MuJoCo).

7. Project 16 — Tool–tissue contact detection

Goal: Explore an alternative architecture for tool–tissue interaction (TTI) detection — e.g. U-Net [2] + Depth Anything v2 [1] versus YOLO + Depth Anything v2 — and compare it with the provided baseline.

Description: The reference pipeline detects and segments the instrument (YOLOv11-Seg), estimates a monocular depth map (Depth Anything v2 [1]), and isolates the tool–tissue contact region by fusing segmentation with depth (a ViT+depth stage). The project swaps one or more stages (e.g. a U-Net segmentation front-end) and quantifies the effect.

Your task:

- Reproduce and run the provided baseline.
- Implement the alternative architecture (e.g. U-Net+depth vs YOLO+depth front-ends).
- Compare detection/segmentation quality and contact-isolation accuracy on the provided dataset, with a clear metric protocol.

What we provide: annotated dataset and baseline code.

Best suited for: students who already have, or wish to develop, a solid background in machine learning.

Learning objectives: segmentation architectures; use of a monocular-depth foundation model as a fixed component; controlled architecture ablation with a fair evaluation protocol.

References: [1] (Depth Anything v2), [2] (U-Net), [11] (Ultralytics YOLO11); additional references provided by the instructors.

8. Project 17 — Optical hand tracking

Goal: Replace the Weart TouchDiver tracking — which currently requires wearing the Meta Quest controllers — with optical tracking through the Polaris camera.

Description: The haptic glove's pose is currently obtained from the Meta Quest controllers. The project removes that dependency by tracking the hand/glove optically with an NDI Polaris optical tracker and feeding the pose into the existing simulation.

Your task:

- Design a Polaris-trackable marker arrangement for the hand/glove.
- Use the provided localization and tracking algorithms to recover the hand pose in real time.
- Replace the Meta Quest controller input with the Polaris pose in the simulator and validate latency and accuracy.

What we provide: the Polaris camera, and localization and tracking algorithms.

Best suited for: all students (robotics/tracking oriented).

Learning objectives: optical tracking and rigid-body pose estimation; coordinate-frame calibration between the Polaris frame and the simulation frame; real-time integration and latency budgeting.

Suggested approach: reuse the Polaris→scene calibration methodology from the phantom-registration work; report tracking accuracy and end-to-end latency versus the Meta Quest baseline.

References

[1] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, H. Zhao, "Depth Anything V2," *NeurIPS*, 2024. arXiv:2406.09414. Code: https://github.com/DepthAnything/Depth-Anything-V2

[2] O. Ronneberger, P. Fischer, T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI*, 2015.

[3] M. Raissi, P. Perdikaris, G. E. Karniadakis, "Physics-Informed Neural Networks," *Journal of Computational Physics*, 378, 2019.

[4] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, "Learning nonlinear operators via DeepONet," *Nature Machine Intelligence*, 3, 2021.

[5] S. Wang, S. Sankaran, P. Perdikaris, "Respecting causality for training physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024. arXiv:2203.07404. Code: https://github.com/PredictiveIntelligenceLab/CausalPINNs

[6] S. Wang, H. Wang, P. Perdikaris, "Learning the solution operator of parametric PDEs with physics-informed DeepONets," *Science Advances*, 7(40), 2021.

[7] L. Lu *et al.*, "A comprehensive and fair comparison of two neural operators (DeepONet & FNO)," *Computer Methods in Applied Mechanics and Engineering*, 393, 2022 (operator-learning background).

[8] S. A. Maas, B. J. Ellis, G. A. Ateshian, J. A. Weiss, "FEBio: Finite Elements for Biomechanics," *Journal of Biomechanical Engineering*, 134(1), 2012. https://febio.org

[9] P. Kazanzides, Z. Chen, A. Deguet, G. S. Fischer, R. H. Taylor, S. P. DiMaio, "An open-source research kit for the da Vinci surgical system" (dVRK), *IEEE ICRA*, 2014.

[10] E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," *IEEE/RSJ IROS*, 2012.

[11] Ultralytics, "YOLO11," 2024. Documentation / code: https://docs.ultralytics.com

[12] N. Aghakhani, M. Geravand, N. Shahriari, M. Vendittelli, G. Oriolo, "Task control with remote center of motion constraint for minimally invasive robotic surgery," 2013 IEEE international conference on robotics and automation, 5807-5812